

MHIBE Construction and Proof of Security

1 Properties of MHIBE

. First, we review the properties that MHIBE scheme should have.

1.1 From HIBE to MHIBE

1.1.1 Review of HIBE

An ID of depth k is a vector with k entries, denoted as $(I_1, I_2, \dots, I_k)$. We assume $I_i \in \{0, 1\}$ for $i \in [1, k]$ for ease of explanation. From Fig. 1, there is a hierarchy of identities arranged in a tree where identities correspond to the nodes. Each identity is represented by a vector. A secret key corresponding to a node x can and only can be used to decrypt any ciphertext intended for a node z where z is a node in the subtree rooted at x .

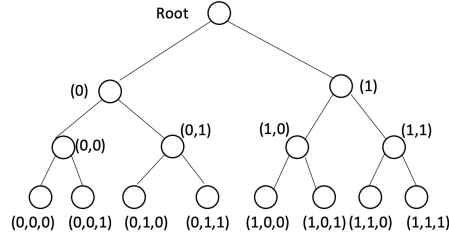


Figure 1: The hierarchy of identities

1.1.2 Properties of MHIBE

A d -dimensional identity consists of d such 1-dimensional identities in HIBE. A d -dimensional identity $ID^{(d)}$ is denoted as

$$\begin{aligned} ID^{(d)} &= (ID_{1,h_1}, \dots, ID_{i,h_i}, \dots, ID_{d,h_d}) \\ &= ((I_{1,1}, I_{1,2}, \dots, I_{1,h_1}), \dots, (I_{i,1}, I_{i,2}, \dots, I_{i,h_i}), \\ &\quad \dots, (I_{d,1}, I_{d,2}, \dots, I_{d,h_d})), \end{aligned}$$

which consists of d vectors each of which denotes an identity as in HIBE. $\forall i \in [1, d]$, $ID_{i,h_i} = (I_{i,1}, I_{i,2}, \dots, I_{i,h_i}) \in (Z_p^*)^{h_i}$ denotes the one-dimensional identity in the i th dimension is at depth h_i . That is, there are d such trees like in Fig. 1. In each dimension i , there is a hierarchy of one-dimensional identities as in HIBE, arranged in the i th tree out of the d trees, as in Fig. 1.

The property of the multi-dimensional hierarchical identity-based encryption scheme (MHIBE) is that, a secret key corresponding to a d -dimensional identity $ID^{(d)} = (ID_{1,h_1}, \dots, ID_{i,h_i}, \dots, ID_{d,h_d})$ can only be used to decrypt any ciphertext intended for a d -dimensional identity $ID'^{(d)} = (ID'_{1,h'_1}, \dots, ID'_{d,h'_d})$ in which for i ranging from 1 to d , ID'_{i,h'_i} is a node in the subtree rooted at ID_{i,h_i} in the i th tree.

1.1.3 Notations

For distinction, from now on, we call d -dimensional identities *d-dimensional identities* and *identities* interchangeably, and we denote the 1-dimensional identities which are components of d -

dimensional identities as *1-dimensional identities* and *domain identities* interchangeably. For example, $ID^{(d)}$ is called a d -dimensional identity which consists of d domain identities, i.e. $ID_{1,h_1}, \dots, ID_{d,h_d}$.

2 MHIBE Construction with JEDI

In [1], the authors propose JEDI scheme where the encryption module works exactly as a two-dimensional HIBE. In their construction, a message is encrypted with a two-dimensional “identity”. Each dimension has a hierarchy. The first dimension is used to represent Uniform Resource Indicator (URI), which is like a file path. For example, a message m can be sent (in the encrypted form) to the URI *buildingA/floor2/roomLHall/sensor0/temp*. Anyone who has a secret key for the *prefix* of this URI can derive a secret key for it to decrypt the message m . Another dimension is used to express time, which is represented as *year/month/day/hour*. Similarly, anyone who has a secret key for the *prefix* of a time representation can derive a secret key for this time presentation. For example, a secret key for 2019/DEC/10 can be used to derive the secret key for 2019/DEC/10/9. In JEDI, the secret key is associated with both URI and time. To be consistent with MHIBE, we can regard the secret key is associated with $(URI, time)$ which can be seen as a two-dimensional identity. The slashes in URI and time representation can be replaced by commas to transform the representation to vectors. As in MHIBE, a secret key associated with the URI URI and the time t (i.e. (URI, t)) can and only can derive the secret key associated with URI' and t' (i.e. (URI', t')) where URI is a *prefix* of URI' and t is a *prefix* of t' .

With JEDI, we can directly extend their encryption scheme to support d -dimensional HIBE where $d \geq 2$. Indeed, the encryption scheme in JEDI is based on WKD-IBE [2] scheme, which is also the core of the MHIBE construction. Thus, we first explain WKD-IBE [2] scheme as a preparation for our construction.

2.1 WKD-IBE scheme

In WKD-IBE scheme, messages are encrypted with patterns. We assume all patterns have a fixed length l . A pattern P with length l is a list of values: $P = (\mathbb{Z}_p^* \cup \{\perp\})^l$, where $\perp$ means a wildcard. We denote $P(i)$ as the i th component of P , 1-indexed. For consistency with MHIBE construction, we regard P as a vector of l entries, where the i th entry corresponds to $P(i)$.

Here, we denote the component containing $\mathbb{Z}_p^*$ as *fixed*, and the component containing $\perp$ as *free*. A pattern P_2 matches a pattern P_1 if and only if: for all $i \in [1, l]$, $P_1(i) = \perp$ or $P_1(i) = P_2(i)$. That means, if the i th component is *fixed* in P_1 , P_2 must match it at the index i . If the i th component is *free* (i.e. $\perp$), whatever $P_2(i)$ is, it can match $P_1(i)$ since $P_1(i)$ is a wildcard.

A secret key associated with a pattern P_1 can decrypt the message encrypted with a pattern P_2 if and only if $P_1 = P_2$. Furthermore, in WKD-IBE, a secret key associated with a pattern P_1 can be used to derive the secret key for a pattern P_2 where P_2 matches P_1 . Thus, a secret key associated with a pattern P_1 can be used to decrypt any message encrypted with a pattern P_2 if P_2 matches P_1 .

The concept of *fixed* and *free* can help the readers understand the process of deriving secret keys from one pattern P_1 to another pattern P_2 . We define them as follows.

Definition 1. For a pattern P , we define:

$$\begin{aligned} \text{fixed}(P) &= \{(i, P(i)) | P(i) \neq \perp\} \\ \text{free}(P) &= \{i | P(i) = \perp\} \end{aligned}$$

Thus, we can regard the process of deriving secret keys as replacing the $free(P)$ in the pattern P with $\mathbb{Z}_p^*$ to generate secret keys for new patterns.

2.2 Supporting Multiple Dimensions Using WKD-IBE

Recall that JEDI realizes a 2-dimensional HIBE scheme where the first dimension represents URI and the second dimension represents time. They achieve the 2-dimensional HIBE by using WKD-IBE creatively. Specifically, assume the maximum length of URI is l_1 and the maximum length of a time representation is l_2 . JEDI sets the maximum length of a pattern to be $l = l_1 + l_2$. A pattern P is associated with the URI and time (P a vector) where the first l_1 entries represent URI and the last l_2 entries represent the time.

The message is encrypted with a pattern. Here, we explain how to represent URI and time as patterns in encryption. If the length of the URI associated with m is shorter than l_1 , for example, $l_1 = 5$ and URI is $a/b/c$, the URI will be represented as $a/b/c/\$/\perp$ where $\$$ is a terminator symbol and $\perp$ means this entry is free. For the time, if the length of the time representation is shorter than $l_2 = 4$, such as $2019/DEC/$, it will be represented as $2019/DEC/\perp/\perp$ which means the last two entries are free. Thus, a message associated with $a/b/c/$ and $2019/DEC/$ will be encrypted with the pattern $(a, b, c, \$, \perp, 2019, DEC, \perp, \perp)$.

Secret keys are also associated with patterns. If the length of URI or of time representation is shorter than the maximum length, the remaining non-fixed entries are automatically regarded as $\perp$, which means they are free. For example, a secret key associated with $a/b/c/$ and $2019/DEC/$ is equivalent that it is associated with the pattern $(a, b, c, \perp, \perp, 2019, DEC, \perp, \perp)$.

Thus, the URI and time can be combined to form a pattern P_2 as in WKD-IBE and the message m can be encrypted with P_2 . A secret key associated with a pattern P_1 can be used to derive a secret key for the pattern P_2 if and only if P_2 matches P_1 . Since URI and time representation fix their representation slots from left to right, P_2 matches P_1 also means the URI in P_1 is a prefix of the URI in P_2 and the time in P_1 is also a prefix of that of P_2 .

Note that adding a terminator symbol $\$$ in URI in encryption is not necessary for security purpose. Thus, we can omit $\$$ and replace it with $\perp$ in our construction.

2.3 MHIBE construction with JEDI

We directly extend the encryption scheme in JEDI to support d -dimensional HIBE where $d \geq 2$. We let the maximum length of a pattern to be $l = dl_b$ where l_b is the length of binary strings to represent keys in the key-value store $\mathcal{D}$ (l_b corresponds to the l in the one-dimensional scheme [3]). We explain our detailed construction which is based on the construction provided in JEDI scheme [1], Appendix A.1.

As the first step, as in JEDI where they show how to represent the $(URI, time)$ as patterns, we first describe how to represent the d -dimensional identities as patterns.

2.3.1 Representing d -dimensional identities as patterns

We define an algorithm $Transform(ID^{(d)})$ which takes a d -dimensional identity as input, and transforms it to a pattern P as output.

- $Transform(ID^{(d)})$. Given a d -dimensional identity $ID^{(d)} = (ID_{1,h_1}, \dots, ID_{i,h_i}, \dots, ID_{d,h_d})$,

$ID^{(d)}$ is parsed as

$$\begin{aligned} & (ID_{1,h_1}, \dots, ID_{i,h_i}, \dots, ID_{d,h_d}) \\ &= ((I_{1,1}, I_{1,2}, \dots, I_{1,h_1}), \dots, (I_{i,1}, I_{i,2}, \dots, I_{i,h_i}), \\ & \quad \dots, (I_{d,1}, I_{d,2}, \dots, I_{d,h_d})) \end{aligned}$$

Like JEDI, we would like each domain identity to have the maximum length l_b (we will explain this setting later in the detailed construction) since the maximum number of entries are designed specifically to represent identities in this dimension. Thus, for $i \in [1, d]$, if the length of ID_{i,h_i} (i.e. h_i) is smaller than l_b , the algorithm pads the remaining $l_b - h_i$ entries as $\perp$, meaning they are free. Thus, it is equivalent that the $ID^{(d)}$ is a pattern P as

$$\begin{aligned} P &= ID^{(d)} \\ &= ((I_{1,1}, I_{1,2}, \dots, I_{1,h_1}, \underbrace{\perp, \dots, \perp}_{l_b - h_1}), \dots, (I_{i,1}, I_{i,2}, \dots, I_{i,h_i}, \underbrace{\perp, \dots, \perp}_{l_b - h_i}), \\ & \quad \dots, (I_{d,1}, I_{d,2}, \dots, I_{d,h_d}, \underbrace{\perp, \dots, \perp}_{l_b - h_d})) \\ &= (I_1, I_2, \dots, I_{h_1}, \underbrace{\perp, \dots, \perp}_{l_b - h_1}, \dots, I_{(i-1)l_b+1}, I_{(i-1)l_b+2}, \dots, I_{(i-1)l_b+h_i}, \underbrace{\perp, \dots, \perp}_{l_b - h_i}, \dots, \\ & \quad I_{(d-1)l_b+1}, I_{(d-1)l_b+2}, \dots, I_{(d-1)l_b+h_d}, \underbrace{\perp, \dots, \perp}_{l_b - h_d}), \end{aligned}$$

We get the last equation by numbering the entries (i.e. I_i ($i \in [1, dl_b]$)) based on their indices in the vector that represents the pattern P .

After transforming the d -dimensional identity $ID^{(d)} = (ID_{1,h_1}, \dots, ID_{i,h_i}, \dots, ID_{d,h_d})$ to the pattern P , we have the following definition which is an equivalent expression for P' matches P .

Definition 2. Given $ID^{(d)} = (ID_{1,h_1}, \dots, ID_{i,h_i}, \dots, ID_{d,h_d})$ where $\text{Transform}(ID^{(d)}) = P$ and $ID'^{(d)} = (ID'_{1,h'_1}, \dots, ID'_{d,h'_d})$ where $\text{Transform}(ID'^{(d)}) = P'$. We denote $ID'^{(d)}$ matches $ID^{(d)}$ iff $i \in [1, d]$, ID_{i,h_i} is a prefix of ID'_{i,h'_i} .

Iff P' matches P , for $i \in [1, d]$, ID_{i,h_i} is a prefix of ID'_{i,h'_i} . Thus, $ID'^{(d)}$ matches $ID^{(d)}$ is equivalent to P' matches P .

More importantly, by this definition, we can translate P' matches P in WKD-IBE scheme to $ID'^{(d)}$ matches $ID^{(d)}$ thus construct our MHIBE scheme. To be specific, in WKD-IBE scheme, the secret key associated with P should be able to derive a secret key associated with P' if and only if P' matches P . Correspondingly, a secret key associated with $ID^{(d)}$ (corresponds to P) can be used to derive the secret key for $ID'^{(d)}$ (corresponds to P') if and only if $ID'^{(d)}$ matches $ID^{(d)}$. Thus, by transforming identities to patterns, we can build MHIBE scheme directly on the basis of WKD-IBE scheme.

2.3.2 Detailed construction

The construction is based on bilinear groups. Let $\mathbb{G}_1, \mathbb{G}_2$ and $\mathbb{G}_T$ be cyclic groups of prime order p , and let e be a bilinear map such that $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$. The security parameter is κ . The construction consists of four phases : *Setup*, *KeyGen*, *Encrypt* and *Decrypt*.

We denote the MHIKE identity as a d -dimensional identity $ID^{(d)} = (ID_{1,h_1}, \dots, ID_{i,h_i}, \dots, ID_{d,h_d})$ where for $i \in [1, d]$, h_i is the length of the 1-dimensional identity in the i th dimension and the h_i entries are fixed. Assume the length of binary strings to represent keys in the key-value store $\mathcal{D}$ is l_b (recall that our scheme is the extension of [3] to multiple keys). Then, in the *Setup* phase, the algorithm sets $l = dl_b$.

- *Setup*(1^l). Select $g \xleftarrow{\$} \mathbb{G}_2$ and $\alpha \xleftarrow{\$} \mathbb{Z}_p^*$. Let $g_1 = g^\alpha$. Select $g_2, g_3, h_1, h_2, \dots, h_l \xleftarrow{\$} \mathbb{G}_1$. Set $g_4 = g_2^\alpha$. The public parameters are $Params = (g, g_1, g_2, g_3, h_1, h_2, \dots, h_l)$. The master secret key is set as $MSK = g_4$.

For ease of exposition, we divide parameters from h_1 to h_l into d segments where the i th segment contains l_b parameters from $h_{(i-1)l_b+1}$ to h_{il_b} . This division can help serve d -dimensional identities for MHIKE, where each segment can be used for a one-dimensional identity, respectively.

- *KeyGen*($K, ID^{(d)}$). First, the algorithm transforms the d -dimensional identity to a corresponding pattern by $P \leftarrow Transform(ID^{(d)})$. Then we explain how to generate secret keys for the pattern P .
 - If K is the master secret key, i.e. $K = g_4$, the algorithm generates the secret key for $ID^{(d)}$ using the corresponding P as:

$$a_1 = g_4 \left(g_3 \prod_{(i, P(i)) \in fixed(P)} h_i^{I_i} \right)^r; a_2 = g^r, b = \{(i, b_i = h_i^r)\}_{i \in free(P)},$$

where $r \xleftarrow{\$} \mathbb{Z}_p^*$.

- If K is not the master secret key, K must be a secret key associated with a d -dimensional identity $ID_K^{(d)}$ which corresponds to a pattern P_K . If P matches P_K , which means each one-dimensional identity in $ID_K^{(d)}$ is a prefix of that in $ID^{(d)}$, the algorithm will output a secret key associated with $ID^{(d)}$. Otherwise, the algorithm will return $\perp$ which means it will end without outputting a secret key.

If P matches P_K , parse K as (a_1, a_2, b) as above. The algorithm generates the secret key (a'_1, a'_2, b') for P using K as:

$$\begin{aligned} a'_1 &= a_1 \cdot \left(g_3 \prod_{(i, I_i) \in fixed(P)} h_i^{I_i} \right)^{r'} \cdot \left(\prod_{(i, I_i) \in fixed(P), i \in free(P_K)} b_i^{I_i} \right); \\ a'_2 &= a_2 \cdot g^{r'}, b' = \{(i, b'_i = b_i \cdot h_i^{r'})\}_{i \in free(P)}, \end{aligned}$$

where $r' \xleftarrow{\$} \mathbb{Z}_p^*$.

- Note that the resulting key is identically distributed, regardless of whether or not the input key K is the master secret key. This is important when we use MHIKE for zero-knowledge proof. This property is referred to as history-independence in [1], Appendix.E.
- *Encrypt*($ID^{(d)}, m$). To encrypt a message $m \in \mathbb{G}_T$ for a d -dimensional identity $ID^{(d)}$, the algorithm first transform the identity to a pattern $P \leftarrow Transform(ID^{(d)})$. Then, with P , the algorithm chooses $t \in \mathbb{Z}_p^*$ and outputs the ciphertext $C = (C_1, C_2, C_3) \in \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_T$, where

$$C_1 = g^t, C_2 = \left(g_3 \prod_{(i, I_i) \in fixed(P)} h_i^{I_i} \right)^t; C_3 = m \cdot e(g_1, g_2)^t$$

- *Decrypt*(K, C). Let $C = (C_1, C_2, C_3)$ and P be the pattern corresponding to the $ID^{(d)}$ with which the ciphertext is encrypted. If K is the master secret key g_4 , the message can be decrypted by computing $C_3/e(C_1, K)$. Otherwise, parse K as (a_1, a_2, b) . If K is associated with a pattern P_K (i.e. d -dimensional identity $ID_K^{(d)}$) and P matches P_K but $P \neq P_K$, the decryption algorithm first computes the secret key for P by calling $KeyGen(K, ID^{(d)})$ which outputs a secret key K' associated with $ID^{(d)}$. Parse K' as (a'_1, a'_2, b') . Then, the message is recovered by the following computation.

$$m \leftarrow C_3 \cdot \frac{e(a'_2, C_2)}{e(C_1, a'_1)}$$

If P matches P_K and $P = P_K$, the process of calling $KeyGen(K, ID^{(d)})$ can be skipped and the input (a'_1, a'_2, b') for the equation is replaced by (a_1, a_2, b) .

2.4 Proof of security and the history-independence property

Our scheme is a direct application of JEDI [1] without modification. For the history-independence property, the proof is just as the proof in [1]. For the security, we will provide the proof in the following.

To be noted, after reading the proof of security in JEDI scheme [1], I found their proof (Theorem 1 in [1]) was flawed. The authors observe that the encryption algorithm in the original WKD-IBE scheme in [2] which can only work with identities, can be extended to work with arbitrary patterns. Thus, they construct their own WKD-IBE scheme on the basis of the original WKD-IBE scheme using their observation. However, they did not provide a proof for their observation. In their proof of security, they prove the security of their JEDI directly on the basis of their constructed WKD-IBE rather than the original WKD-IBE scheme, which is not rigorous.

Our scheme is constructed on the basis of the WKD-IBE scheme in JEDI scheme [1]. However, in the following proof, I will prove the security of our scheme directly on the basis of the original WKD-IBE scheme in [2].

3 Security of M-HIBE

Theorem 1. *Assume the BBG-WKD-IBE scheme in [2] is IND-sWKID-CPA secure. Then, no probabilistic polynomial-time adversary $\mathcal{A}$ can win the following security game against a challenger $\mathcal{C}$ with non-negligible advantage.*

- **Initialization.** *The adversary $\mathcal{A}$ first outputs a d -dimensional identity*

$$ID^{(d)*} = (ID_{1,h_1}^*, \dots, ID_{i,h_i}^*, \dots, ID_{d,h_d}^*)$$

that it intends to attack.

- **Setup.** *The challenger $\mathcal{C}$ runs the Setup algorithm and gives $\mathcal{A}$ the resulting system parameters, keeping the master secret key to itself.*
- **Phase 1.** *$\mathcal{A}$ issues q_m secret key queries corresponding d -dimensional identities $ID_1^{(d)}, \dots, ID_{q_m}^{(d)}$. The restriction is $ID^{(d)*}$ matches none of them.*

- **Challenge.** Once $\mathcal{A}$ decides that Phase 1 is over, it outputs two equal length plaintexts $M_0, M_1 \in M$ on which it wishes to be challenged. $\mathcal{C}$ picks a random bit $b \in \{0, 1\}$ and sets the challenge ciphertext to $CT = \text{Encrypt}(\text{params}, ID^{(d)*}, M_b)$, which is sent to $\mathcal{A}$.
- **Phase 2.** $\mathcal{A}$ issues secret key queries corresponding to the d -dimensional identities $ID_{q_m+1}^{(d)}, ID_{q_K}^{(d)}$ as long as the $ID^{(d)*}$ does not match any of them.
- **Guess:** Finally, the adversary outputs a guess $b' \in \{0, 1\}$ and wins if $b = b'$. We define the advantage of the adversary $\mathcal{A}$ in attacking the scheme $\mathcal{E}$ as

$$\text{Adv}_{\mathcal{E}, \mathcal{A}} = |P[b' = b] - 1/2|$$

4 Proof of Theorem 1

Proof. We show that, given an adversary $\mathcal{A}$ with non-negligible advantage in the game in Theorem 1, one can construct an algorithm $\mathcal{B}$ with non-negligible advantage in the IND-sWKID-CPA security game [2]. We denote as $\mathcal{C}$ the IND-sWKID-CPA security challenger.

- **Initialization** Adversary $\mathcal{A}$ first outputs to $\mathcal{B}$ a d -dimensional identity $ID^{(d)*} = (ID_{1, h_1}^*, \dots, ID_{d, h_d}^*)$ that it intends to attack. By transforming this identity to the corresponding pattern P^* , the length of P^* is dl . Assume $\text{fixed}(P)$ is the set of indices where the entries are not wildcards. Since in the BBG-WKD-IBE scheme, the adversary should choose an identity rather than a pattern to attack, $\mathcal{B}$ replaces those wildcards in P (the indices are in $\text{fixed}(P)$) with 0. We denote the resulting identity which keeps the non-wildcard entries in P^* and replaces wildcards with 0s as P'^* . Thus, $\mathcal{B}$ outputs P'^* to the BBG-WKD-IBE scheme.
- **Setup** The BBG-WKD-IBE challenger outputs $mpk = (g, g_1, g_2, g_3, h_1, \dots, h_{dl_b})$ to $\mathcal{B}$ and keeps $msk = g_4$. Then, $\mathcal{B}$ output $mpk = (g, g_1, g_2, g_3, h_1, \dots, h_{dl_b})$ to the adversary $\mathcal{A}$.
- **Phase 1.** The adversary $\mathcal{A}$ issues q_m queries corresponding to any d -dimensional identity $ID_i^{(d)} (i \in [1, q_m])$ as long as $ID^{(d)*}$ does not match any of them. To be noted, for those queries, neither of their entries can be 0 since in our construction, we do not allow zero entries in domain identities. Thus, we discard d -dimensional identities with zero entries. After receiving those queries, $\mathcal{B}$ just transfer those d -dimensional identity to patterns and queries secret key corresponding to those patterns from $\mathcal{C}$. Since $ID^{(d)*}$ does not match any of $ID_i^{(d)} (i \in [1, q_m])$, the transformed and modified pattern P'^* also does not match any of those patterns associated with the queried identities. (Imagine in each pattern P_j corresponding to the d -dimensional identity $ID_j^{(d)}$, there is some indice k_j such that $P^*(k_j) \neq P_j(k_j)$ where $j \in [1, q_m]$.) The BBG-WKD-IBE challenger generate secret keys for P_j which is associated with $ID_j^{(d)}$ as:

$$a_1 = g_4(g_3 \prod_{(i, I_i) \in \text{fixed}(P_j)} h_i^{I_i})^r; a_2 = g^r, b' = \{b_i = h_i^r\}_{i \in \text{free}(P_j)}$$

The BBG-WKD-IBE challenger returns the above secret key to $\mathcal{B}$ and $\mathcal{B}$ returns it to $\mathcal{A}$ without any modification except adding the indices of free entries in b' .

- **Challenge.** When the adversary decides phase 1 is over, it output two messages M_0, M_1 on which they want to be challenged on. $\mathcal{B}$ sends them directly to the BBG-WKD-IBE

challenger. The challenger flips a coin b , and the output ciphertext is

$$C_1^* = g^t, C_2^* = \left(g_3 \prod_{i=1}^{dl_b} h_i^{I_i^*}\right)^t; C_3^* = M_b \cdot e(g_1, g_2)^t$$

To be noted, for those entries that are padded with 0s, $h_i^{I_i^*}$ is 1. Thus,

$$C_2^* = \left(g_3 \prod_{i=1}^{dl_b} h_i^{I_i^*}\right)^t = \left(g_3 \prod_{(i, I_i^*) \in \text{fixed}(P^*)} h_i^{I_i^*}\right)^t$$

. $\mathcal{B}$ can directly return this ciphertext to $\mathcal{A}$.

- **Guess.** $\mathcal{A}$ outputs a guess b' . Based on $\mathcal{A}$'s guess, $\mathcal{B}$ output the same b' as his/her guess. We can see that the advantage of the adversary $\mathcal{B}$ in attacking the BBG-WKD-IBE scheme $\mathcal{E}'$ is no less than the advantage of the adversary $\mathcal{A}$ in attacking our scheme $\mathcal{E}$. That is,

$$\text{Adv}_{\mathcal{E}, \mathcal{A}} = |P[b' = b] - 1/2| \leq \text{Adv}_{\mathcal{E}', \mathcal{B}} = |P[b' = b] - 1/2|$$

If $\text{Adv}_{\mathcal{E}, \mathcal{A}}$ is no less than some non-negligible probability, there is an adversary $\mathcal{B}$ with non-negligible advantage to break IND-sWKID-CPA security in BBG-WKD-IBE scheme.

References

- [1] S. Kumar, Y. Hu, M. P. Andersen, R. A. Popa, and D. E. Culler, “Jedi: Many-to-many end-to-end encryption and key delegation for iot,” *arXiv preprint arXiv:1905.13369*, 2019.
- [2] M. Abdalla, E. Kiltz, and G. Neven, “Generalized key delegation for hierarchical identity-based encryption,” in *European Symposium on Research in Computer Security*. Springer, 2007, pp. 139–154.
- [3] E. Ghosh, O. Ohrimenko, and R. Tamassia, “Efficient verifiable range and closest point queries in zero-knowledge,” *Proceedings on Privacy Enhancing Technologies*, vol. 2016, no. 4, pp. 373–388, 2016.